



ELKATECH LAUNCHPAD

App Flow & UI/UX Flow

ElkaTech Launchpad — Service Platform

Document: App Flow & UI/UX Flow

Project: ElkaTech Launchpad

Version: v2.0

Date: 11 June 2026

Status: Current — feature/owner-support-roles

Table of Contents

1. 1. Overview

2. 2. Route Map

– Public and auth routes

– Protected portal routes

3. 3. Sidebar Navigation by Role

4. 4. Public Visitor Flow

5. 5. Customer Authentication Flow

6. 6. Customer Service-Profile and Request Flow

7. 7. Admin Flow

8. 8. Customer Machines Flow

9. 9. Engineer Flow

10. 10. Owner / Support Flow

11. 11. My Account Flow

12. 12. Important Empty States

13. 13. Key UX Decisions

14. 14. Responsive Behavior Summary

1. Overview

This document describes how users move through ElkaTech Launchpad — the public website and the authenticated service portal — and the key UI/UX decisions behind each flow. Routes, navigation, and access rules below reflect the current branch (`apps/web/src/App.tsx` , `PortalShell.tsx` , and the portal pages).

2. Route Map

Public and auth routes

ROUTE	PURPOSE
<code>/</code>	Landing page
<code>/solvent-printers</code> , <code>/uv-printers</code> , <code>/laser-cutting-machines</code>	Category pages
<code>/lamination-machines</code> , <code>/desktop-uv-printer</code> , <code>/inkjet-printer</code> , <code>/flatbed-uv-printer</code>	Category pages
<code>/login</code> , <code>/signup</code>	Sign in, sign up / accept invite
<code>/forgot-password</code> , <code>/reset-password</code> , <code>/verify-email</code>	Account recovery and verification

Protected portal routes

ROUTE	ACCESS
<code>/app/requests</code>	All authenticated users
<code>/app/requests/new</code>	All authenticated users (customer vs staff flow)
<code>/app/requests/:id</code>	Users with access to that request
<code>/app/complete-profile</code>	Customers completing onboarding
<code>/app/account</code>	All authenticated users
<code>/app/queue</code>	engineer, support, owner, admin
<code>/app/support</code>	support, owner, admin
<code>/app/customer-activity</code> (+ <code>/:customerId</code>)	support, owner, admin
<code>/app/admin</code>	admin
<code>/app/users</code>	owner, admin

ROUTE	ACCESS
<code>/app/machines</code> (+ <code>/:customerId</code>)	owner, admin

3. Sidebar Navigation by Role

The `PortalShell` sidebar is built from the same permission helpers the backend uses, so each role sees only what it can use.

NAV ITEM	CUSTOMER	ENGINEER	SUPPORT	OWNER	ADMIN
Requests	Yes	Yes	Yes	Yes	Yes
Create Request	Yes	Yes	Yes	Yes	Yes
Queue	–	Yes	Yes	Yes	Yes
Support	–	–	Yes	Yes	Yes
Customer Activity	–	–	Yes	Yes	Yes
Customer Machines	–	–	–	Yes	Yes
Users	–	–	–	Yes	Yes
Admin	–	–	–	–	Yes
My Account	Yes	Yes	Yes	Yes	Yes

The sidebar is collapsible (state persisted), includes a theme selector, a profile block, and logout, and collapses to a mobile drawer below the `lg` breakpoint.

4. Public Visitor Flow

1. Visitor lands on `/`, scrolls hero → about → work → brands → infrastructure → why-us → contact → footer.
2. Visitor opens a product category, browses the carousel, specifications, and brochure links.
3. Visitor enters the portal via the header/CTA, reaching `/login` or `/signup`.

Key UX: persisted light/dark/system theme; theme-aware header variants for readable contrast on dark landing vs light category pages; layout-stable reveals that never shift on refresh.

5. Customer Authentication Flow

1. Sign up (Firebase email/password or Google) or accept a staff invite.
2. Email verification — a friendly notice appears until verified.

3. Approval — new public customers start in `pending_approval` and see a polished pending/rejected/suspended state card; they cannot create requests yet.
4. After approval and verification, the customer is routed to complete their profile if it is incomplete.

6. Customer Service-Profile and Request Flow

1. `/app/complete-profile` collects required company/contact/address fields; the backend re-checks completion before allowing request creation.
2. The customer opens `/app/requests/new`, which loads their **active** machines (customer-safe view; no internal serials).
3. They pick a machine, choose an issue type (e.g. "Printing issue", "Ink issue"), set urgency, describe the problem, and optionally attach photos/video.
4. Attachments upload directly to R2 via a presigned URL; the request is filed under the customer's account with product/site/serial copied from the machine.
5. The customer tracks status, exchanges customer-visible messages, and views the resolution on `/app/requests/:id`.

Empty states: "No machines are linked to your account yet" guides the customer to contact ElkaTech; an unverified or unapproved account sees a dedicated state card instead of the form.

7. Admin Flow

1. `/app/admin` shows approval counts, request load, recent activity, and live service heartbeats.
2. `/app/users` lists users with approve/reject, suspend/reactivate, role change, and (admin-only) permanent removal; staff invites are issued here.
3. `/app/machines` manages the global customer-machine inventory; the detail page `/app/machines/:customerId` is the per-customer machine and request workspace.
4. Admin can create requests on behalf of customers and manage the queue.

8. Customer Machines Flow

1. The machines dashboard lists assignments with customer/product/status filters.
2. A link/edit dialog assigns a catalog product to a customer with a product snapshot, labels, site, dates, and internal serial.
3. The machine profile/detail view shows the machine plus its request history; a detail drawer surfaces machine context inline from the users page.
4. Machines are deactivated (soft `inactive`) and can be reactivated; history is preserved.

9. Engineer Flow

1. `/app/queue` shows open and assigned requests.
2. The engineer claims an unassigned request or works one already assigned.
3. They update status along the allowed transitions, message the customer (customer-visible) or leave internal notes, and resolve the request.

10. Owner / Support Flow

- **Support dashboard** (`/app/support`): KPI cards (in-queue, assigned, in progress, waiting, resolved, unassigned), a filterable request table (status tabs, unassigned-only, search), and an **assign-engineer modal** that lists engineers only and refetches on success.
- **Customer Activity** (`/app/customer-activity`): KPI cards (total/active customers, open/pending/resolved requests, customers with machines), search and status filters, a customer table, and a detail drawer (profile, machines, and clickable request history) with a "Create request for this customer" action.
- **Owner** additionally reaches Users and Customer Machines, can approve/suspend users and assign non-admin roles, but never sees delete controls or the system Admin panel.

11. My Account Flow

1. `/app/account` shows the profile summary and allows a display-name/profile update (never role/email/password/approval via this path).
2. The user can trigger a self password reset email.
3. Email/account-state changes are handled through the secure auth flows; sensitive account fields are not editable directly from this page.

12. Important Empty States

- No linked machines (customer create-request).
- No customers available (staff create-on-behalf).
- No engineers exist (assign modal shows a clean prompt to create one).
- Empty queue / no matching requests after filtering.
- Pending/rejected/suspended account state cards.

13. Key UX Decisions

- Hidden controls always correspond to a backend `403` — the UI never implies an action a role cannot perform.

- Machine-scoped request creation removes free-typing of product/serial/location for customers, reducing mis-routed requests.
- Destructive actions are visually distinct and restricted; permanent deletion is admin-only and absent from owner/support UI.
- Status and priority are shown as consistent color-coded badges across pages.

14. Responsive Behavior Summary

- Desktop: collapsible sidebar, compact dense tables, aligned filters, and compact action buttons.
- Tablet/mobile: the sidebar becomes a drawer with a top bar; wide tables scroll horizontally within their card so content is never clipped; dashboards and the activity views remain readable.
- Motion respects `prefers-reduced-motion`; entrance reveals stay layout-stable.